END - END COMMUNICATION AT TRANSPORT LAYER

a) Implement echo client server using TCP/UDP sockets

AIM:

To implement an echo client-server application using TCP sockets and verify communication between the client and server

STEPS, PROCEDURE:

1. Create a TCP Echo server using server ~~socket~~ and socket
2. Assign a port number to the server.
3. Compile and execute the TCP server program
4. Create a TCP Echo client using socket
5. Connect the client to the server using the server address and port number.
6. Enter a message through the client
7. The server receives the message and sends it back to the client.
8. Display the echoed message at the client
9. Enter byte to terminal the communication.

RESULT:

Thus, the TCP Echo client-server application was successfully implemented and the client successfully received the echoed message from the server

PROGRAM:

TCP ECHOSERVER.JAVA

```
import java.io.*;
import java.net.*;
public class TCPEchoServer{
    public static void main (String [] args)
throws Exception{
    ServerSocket ss = new ServerSocket (5000);
    Socket s= ss.accept();
    BufferedReader in = new BufferedReader;
    String msg;
    while ((msg = in.readline()) != null){
     out.println ("Echo"+ msg);
     if (msg.equalsIgnoreCase("bye")) breaks;
  }
    s.close();
    ss.close(); }
}
```

TCP ECHO CLIENT.JAVA:

```
import java.io.*;
import java.net.*;
public class TCPEchoClient{
    public static void main (String [] args ){
        Socket s: new socket ("local host", 5000);
        BufferReader in = new BufferedReader(
          printWriter out = new PrintWriter (s.getOutputStream(), true
        BufferReader kb = new BufferedReader;
       String msg;
       while (! (msg = kb.readLine()).equals
       Ignore case ("bye")){
            out.println (msg);
            System.out.println (in.readline());
        }
        s.close();
      }
  }
```

b) Echo Client - Server using UDP

AIM:

To Implement an Echo client-server application using UDP sockets and verify communication between the client and server

PROCEDURE:

1. Create a UPD Echo server using Datagram socket and Datagram packet
2. Assign a port number to the UDP server
3. Compile and execute the UDP server program
4. Create a UDP Echo client using Datagram Socket and Datagram packet
5. Specify the server address and port number.
6. Enter a message through the client.
7. The client sends the message as a UDP datagram to the server.
8. The server receives the datagram and sends the echoed message back.
9. Display the received echoed message at the client
10. Enter byte to terminate the communication

RESULT:

Thus, the UPP Echo client - server application was successfully implemented and the client successfully received the echoed message from the server.

b) PROGRAM:

UDP ECHOSERVER.JAVA:

```
import java.net.*;
public class UDPEchoServer {
    public static void main (String [] args){
throws Exception {
        DatagramSocket S= new DatagramSocket (6000);
        while (true){
            DatagramPacket P = new DatagramPacket (b, b length),
                                                    s.receive (P);
            S.send (new DatagramPacket (r, r.length,
                P.getAddress(), P.getPort()));
        if (msg.equalsIgnoreCase ("bye")) break; }
    S.close(); }
}
```

UDP ECHOCLIENT.JAVA:

```
import java.net.*;
import java.io.*;
public class UDPEchoClient {
    public static void main (String [] args)
    throws Exception{
        DatagramSocket S= new DatagramSocket();
        InetAddress a = InetAddress.getByName ("Localhost");
        String msg;
        while (!(msg = kb.readLine()).equals Ignorecase ("bye")) {
            S.send (new DatagramPacket (b, b.length));
            S.receive (P);
            System.out.println (new String ( P.getData(), 0,
                P.getLength()));
        }
        S.close();
    }
}
```

c) Chat client - server using TCP / UDP Sockets.

AIM:

To implement a chat client - server application using sockets and enable communication between multiple clients through a server.

PROCEDURE:

1. Create a chat server program using socket programming
2. Start and run the chat server
3. Create the chat client program and connect it to the server
4. Run two or more client programs in seperate terminals
5. Enter a name of each client.
6. Send messages from one client to the server
7. The server broadcasts the message to the other connected clients.
8. Verify that the clients can exchange message through the server.
9. Continue the communication between the clients
10. Enter /quit to terminates a client connection.

RESULT:

Thus, the chat client - server application was successfully implemented and message was successfully exchanged between multiple clients through the server.

2) PROGRAM:

CHAT SERVER . JAVA:

```
import java.io.*;
import java.util.*;

public class ChatServer{
   static Set<PrintWriter> clients: Collections.synchronizedSet
                                    (new HashSet<>());

   public static void main (String [] args)
   throws Exception{
        ServerSocket ss = newServerSocket (5000);
        System.out.println (" Chat Server started");
        while (true){
            Socket s= ss.accept ();}
       }
       clients.remove (out);
       s.close();
   }
    catch (Exception e){}
}
```

CHAT CLIENT .JAVA:

```
import java.io.*;
import java.net.*;
 Public class chatClient {
    public static void main (String[]args) throws Exception{
        Socket s = new Socket ("127.0.0.1", 5000);
        new thread () ->{
        try{
            string m;
            while ((m = in.readline())!= null)
            System.out. println (m) ; }
         catch (Exception e){}
        string msg;
        while ((msg = kb. readLine ()) ! = null){
            out.println (msg);
            if (msg.equalsIgnoreCase (" /quiet ")) break;
        }
        s.close();
    }
}
```